



SoftClient4ES

BENCHMARK REPORT

Extracting data out of Elasticsearch

SoftClient4ES over Arrow Flight SQL, measured against Trino's Elasticsearch connector — same host, same cluster, same 10-million-row index, back to back.

DATASET

10,000,000

rows · 8 columns

SOFTCLIENT4ES

0.2.5

Arrow Flight SQL sidecar

TRINO

483

Elasticsearch connector

ELASTICSEARCH

8.18.3

single node

1 Summary

This benchmark measures how efficiently each system extracts data **out of Elasticsearch** into a Python data-science client (pandas, polars, DuckDB). Both run against the same single-node Elasticsearch, the same 10-million-row index, on the same host, back to back.

Who this is for: data teams pulling large result sets out of Elasticsearch into Python. It is **not** a distributed-analytics, federation, or SQL-coverage comparison — for those, Trino is the right tool and this benchmark deliberately does not exercise its strengths.

AGGREGATION PUSHDOWN

0 MB vs 1.4 GB

Data moved off the cluster for a `GROUP BY` returning 100 rows. SoftClient4ES compiles it into an Elasticsearch aggregation; Trino scans all 10M rows into its own engine.

CLIENT MEMORY FLOOR

2 GB container

Lands all 10M rows as a DataFrame inside a hard 2 GB cap. Trino's fastest client is killed there; its stock client is killed even at 8 GB.

CONCURRENCY IN AN 8 GB BUDGET

5 vs 0

Five simultaneous 10M-row extractions complete — 50 million rows for +26% wall time — where a single Trino stock client cannot complete one.

10M-ROW EXTRACTION

1.43× faster

37.1 s vs 53.1 s, on 5.5× less client CPU and 5.0× less client memory — because the client consumes Arrow batches instead of building Python objects.

For work Elasticsearch can compute, SoftClient4ES pushes it into the cluster and moves almost no data off it. For large extractions it runs in far less client memory, and it is faster, because it delivers Arrow columnar batches the client consumes directly where Trino's Python client materialises rows into Python objects. These results hold on a **5-shard index read by a 3-node Trino cluster given 1.5× the CPU**, where the extraction gap widens rather than closes (section 7). **Trino remains marginally ahead on cross-index joins**, gains the most from shard parallelism on aggregate-heavy work, and offers capabilities — distributed execution, spill, connector breadth — this benchmark does not exercise (section 8).

All figures in this report were measured on the **released** sidecar image `softnetwork/softclient4es8-arrow-flight-sql:0.2.5` (digest `sha256:d4fb6950f09f...`), Trino 483 and Elasticsearch 8.18.3. No number is published that did not come out of a run of the harness.

2 Environment

SoftClient4ES sidecar	<code>softclient4es8-arrow-flight-sql:0.2.5</code> — Arrow Flight SQL, gRPC :32010
Trino	483 (official image), Elasticsearch connector
Elasticsearch	8.18.3, single node
Index	<code>bench_events_10m</code> — 10,000,000 documents, flat mapping, 1 primary shard, 0 replicas, force-merged (916 MB)
Host	Apple M4 Max, macOS (Darwin arm64), Docker Desktop VM 10 CPU / 15.6 GiB
Container limits	Elasticsearch, sidecar and Trino each capped at 4 CPU / 4 GB (heap 2 GB)
Sensitivity topology (§7)	<code>bench_events_10m_s5</code> — same corpus, same seed, 5 primary shards ; Trino as a 3-node cluster (coordinator + 2 workers, 6 CPU / 8 GB total)
Runs	≥2 warm-ups, then ≥5 measured runs per cell; each run in a fresh client process. Tables show the median and the min-max spread.

Client libraries

Identical versions on both sides wherever a library is shared.

LIBRARY	VERSION	USED BY
CPython	3.12.12 (arm64)	all
pyarrow	25.0.0	SoftClient4ES (Arrow)
adbc-driver-flightsql	1.12.0	SoftClient4ES (Flight SQL)
adbc-driver-manager	1.12.0	ADBC drivers
pandas	3.0.5 (numpy 2.5.1)	both
polars	1.43.2	both (polars variant)
duckdb	1.5.5	both (S2)
trino	0.338.0	Trino (stock client)
sqlalchemy	2.0.51	Trino (<code>pandas.read_sql</code> / <code>pl.read_database</code>)
connectorx	0.4.5	Trino (fast columnar client)
ADBC Trino driver	0.5.1	Trino (ADBC client)

Metrics. *Wall* is end-to-end time including connection. *Client CPU* is `time.process_time()` in the client process. *Peak client memory* is the process's peak physical footprint (`ri_lifetime_max_phys_footprint`), which is immune to the macOS memory compressor. *ES wire* is the bytes that left Elasticsearch, measured from container network counters — sampled at the Elasticsearch container itself, so the figure does not depend on how many containers the engine is made of. Every run asserts the exact expected row count before its timing is recorded; a run that returns the wrong number of rows is discarded, not reported.

3 Scenarios at a glance

ID	QUESTION IT ANSWERS	OUTCOME
S0	What does a naïve Python scroll client cost as a reference floor?	46.5 s to read 10M rows
S1	Extract 10M rows into a client-side columnar table	1.43× faster , 5.5× less CPU, 5.0× less memory
S1r	Extract 10M rows into a pandas / polars DataFrame (what an analyst builds)	42–44% faster on far less CPU and memory
S2	Extract 10M rows and compute an aggregate in DuckDB	1.73× faster , 8.4× less memory
S3	<code>GROUP BY</code> returning 100 rows	0.04 s vs 27 s , and moves 0 vs 1.4 GB off the cluster
S4	Fetch 100 rows (<code>LIMIT 100</code>)	Parity 40 ms vs 57 ms
S5	Does the extraction fit in a constrained-memory container?	Fits 2 GB ; Trino's stock client needs >8 GB, its fastest client 3 GB
S6	How many concurrent extractions fit in an 8 GB budget?	SoftClient4ES 5 (50M rows); Trino 0
J0–J2	Cross-index JOIN landed as a DataFrame	Trino marginally faster on wall (5–9%); SoftClient4ES does 5–20× less client work
§7	Do the results survive a 5-shard index read by a 3-node Trino ?	Yes — S1 gap widens to 1.60× ; client cost and pushdown unchanged; Trino's <code>GROUP BY</code> gains 4.7×

All scenarios run against one 10-million-row index (`bench_events_10m`); the JOIN scenarios also use a 1-million-row index (`bench_1m`).

4 Extraction scenarios

S0 Reference floor — a naïve Python scroll client

A plain Python client that scrolls Elasticsearch and parses each JSON hit. This is not a competitor; it is the cost of the most obvious approach, and it anchors the other numbers.

Result: 46.5 s to read 10M rows (spread 2.8%). Both engines below finish faster than this, because they parse in the JVM and hand the client a columnar buffer rather than JSON.

S1 Extract 10M rows into a columnar client table

The full 10-million-row result set materialised in the client. SoftClient4ES fetches an Arrow table; Trino's stock client fetches rows.

```
# SoftClient4ES - Arrow Flight SQL (adbc_driver_flightsql)
cur.execute("SELECT id, event_ts, amount, qty, status, country, category, name "
            "FROM bench_events_10m")
table = cur.fetch_arrow_table()           # Arrow columnar batches

# Trino - stock client (trino.dbapi)
cur.execute(same_sql)
rows = cur.fetchall()                    # list of Python tuples
```

METRIC	SOFTCLIENT4ES	TRINO	RATIO
Wall median	37.1 s (spread 2.1%)	53.1 s (spread 6.2%)	1.43× faster
Client CPU	4.6 s	25.1 s	5.5× less
Peak client memory	900 MB	4,472 MB	5.0× less
ES wire	2,494 MB	2,926 MB	—
Rows / columns	10,000,000 / 8	10,000,000 / 8	✓

Figure 1 — S1, the cost of landing 10M rows in the client



Each panel has its own scale and unit; bars are comparable within a panel only. Medians of ≥5 runs after 2 warm-ups.

Outcome. SoftClient4ES is faster and far lighter. The reason is client representation: the client consumes Arrow batches without ever building 10 million Python objects, which is what dominates Trino's client CPU and memory.

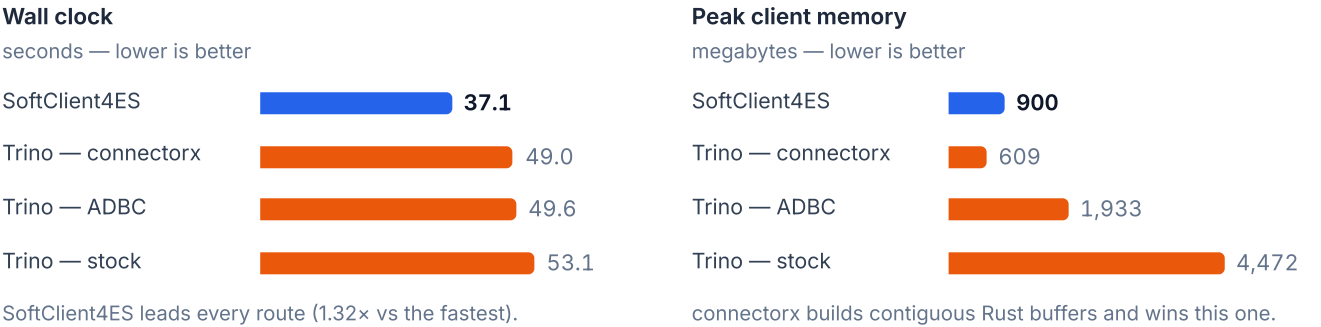
Fairness — measured against Trino's fastest clients

Trino's stock client is not its only option. **connectorx** (a Rust engine that parses Trino's result pages straight into columnar buffers) and the **ADBC Trino driver** both return an Arrow table. Measured on the same query:

ROUTE	WALL	CLIENT CPU	PEAK MEMORY
SoftClient4ES — Arrow Flight SQL	37.1 s	4.6 s	900 MB
Trino — connectorx	49.0 s	9.9 s	609 MB
Trino — ADBC driver	49.6 s	18.9 s	1,933 MB
Trino — stock client	53.1 s	25.1 s	4,472 MB

Figure 2 — S1 against every Trino client

SoftClient4ESTrino clients



The counter-evidence is part of the result. Panels have independent scales.

SoftClient4ES keeps the wall-clock lead against every Trino client (1.32× vs the fastest). On client cost the picture is honest: connectorx, building contiguous buffers in Rust, reaches a lower peak memory (609 MB) than our chunked Arrow batches, and lower CPU than the ADBC driver. **The wall-clock advantage comes from the server and the wire, not from any client library.** Note that 609 MB is a bare Arrow *table*: when the same workflow materialises the whole result as a DataFrame (scenario S5), connectorx needs 2.8 GB where SoftClient4ES needs 1.5 GB — so this memory advantage does not carry to the DataFrame an analyst actually keeps.

S1r

Extract 10M rows into a DataFrame

The artifact a data scientist actually builds. Each side uses its most idiomatic route to a `pandas.DataFrame`, and separately to a `polars.DataFrame`.

```
# SoftClient4ES
df = cur.fetch_arrow_table().to_pandas() # pandas
df = pl.from_arrow(cur.fetch_arrow_table()) # polars

# Trino
df = pandas.read_sql(sql, sqlalchemy_conn) # pandas, via the trino SQLAlchemy dialect
df = pl.read_database(sql, sqlalchemy_conn) # polars
```

To a pandas DataFrame

ROUTE	SOFTCLIENT4ES	TRINO (STOCK)	SOFTCLIENT4ES WIN
Wall	36.9 s	63.7 s	42% faster
Client CPU	4.8 s	36.1 s	7.5× less
Peak memory	1,466 MB	8,317 MB	5.7× less

With Arrow-backed pandas dtypes (`types_mapper=pd.ArrowDtype`), SoftClient4ES needs only 900 MB (38.2 s), versus 7,984 MB for Trino.

To a polars DataFrame

ROUTE	WALL	CLIENT CPU	PEAK MEMORY
SoftClient4ES (<code>pl.from_arrow</code>)	37.7 s	14.9 s	2,487 MB
Trino — connectorx	51.4 s	10.9 s	2,394 MB
Trino — ADBC driver	51.6 s	28.7 s	3,533 MB
Trino — SQLAlchemy	66.4 s	38.1 s	11,052 MB

Outcome. SoftClient4ES lands a DataFrame faster on every route and destination. On the polars destination both sides re-encode Arrow strings into polars' native format, which raises our client CPU; connectorx does the same re-encoding in Rust and comes in slightly lower on CPU and memory. The durable advantage, again, is wall-clock, held on every route.

S2 Extract 10M rows and aggregate in DuckDB

The same fetch, landed in DuckDB and aggregated. SoftClient4ES registers the Arrow table zero-copy; Trino builds a DataFrame from rows and registers that.

```
# SoftClient4ES
con.register("events", cur.fetch_arrow_table())          # zero-copy
con.execute("SELECT category, AVG(amount) FROM events GROUP BY category")

# Trino
df = pandas.DataFrame(cur.fetchall(), columns=cols)
con.register("events", df)
con.execute(same_agg)
```

METRIC	SOFTCLIENT4ES	TRINO	RATIO
Wall median	36.6 s	63.4 s	1.73× faster
Client CPU	5.8 s	34.6 s	5.9× less
Peak client memory	930 MB	7,779 MB	8.4× less

Outcome. The zero-copy Arrow hand-off to DuckDB is the largest memory advantage in the benchmark (8.4×): the columnar result is scanned in place instead of being copied through Python objects.

S3 GROUP BY returning 100 rows

`SELECT category, AVG(amount) FROM bench_events_10m GROUP BY category`, returning 100 groups.

METRIC	SOFTCLIENT4ES	TRINO
Wall median	0.04 s	27.0 s
ES wire	0 MB	1,394 MB
Rows	100	100

Outcome. SoftClient4ES compiles the `GROUP BY` into an Elasticsearch `terms` aggregation: the cluster computes the 100 groups and returns only those 100 rows. Trino's Elasticsearch connector performs predicate push-down only (per its documentation), so it scans all 10M rows into Trino and aggregates there. **This is the clearest architectural difference in the benchmark** — for work Elasticsearch can do, SoftClient4ES does not move the data at all.

S4 Fetch 100 rows (`LIMIT 100`)

The control: when the result is small, the client representation stops mattering.

METRIC	SOFTCLIENT4ES	TRINO
Wall median	0.040 s	0.057 s

Outcome. Near parity, as expected. The extraction advantage only appears at scale, or when work can be pushed into Elasticsearch.

5 Constrained memory and concurrency

S5 Does the extraction fit in a small container?

The client runs inside a container with a hard memory cap (`docker run --memory` , swap pinned equal so the kernel kills rather than pages). The question is binary: does landing 10M rows as a DataFrame complete, or is the process killed?

Whole result set as one DataFrame

CONTAINER CAP	SOFTCLIENT4ES	TRINO (STOCK)	TRINO (CONNECTORX, FASTEST)
8 GB	completes 41 s · 1,531 MB	killed	completes 50 s · 2,908 MB
6 GB	completes 37 s · 1,529 MB	killed	completes 2,909 MB
4 GB	completes 37 s · 1,535 MB	killed	completes 2,907 MB
3 GB	completes 37 s · 1,535 MB	killed	completes 2,910 MB
2 GB	completes 37 s · 1,533 MB	killed	killed

Streaming — neither side holds the whole result (SoftClient4ES via `fetch_record_batch()`, Trino via `pandas.read_sql(chunksize=...)`): both complete at every cap, SoftClient4ES ≈ 38 s / 146 MB, Trino ≈ 59 s / 300 MB.

Outcome. Landing the whole 10M-row DataFrame, SoftClient4ES fits in a **2 GB** container. Trino's stock client is killed even at 8 GB; its fastest client (connectorx) fits at 3 GB but is killed at 2 GB — the cap where SoftClient4ES still completes. Both requirements are independent of the cap (peak memory barely moves across a 4× range), so they reflect the data, not memory thrashing. If the workflow streams instead of materialising, both fit and SoftClient4ES is $\sim 1.6\times$ faster on $\sim 2\times$ less memory.

S6 Concurrent extractions in a fixed memory budget

An 8 GB total client budget, split evenly across N clients launched simultaneously, each extracting 10M rows. How many complete with the correct row count?

ENGINE	N	CAP EACH	COMPLETED	WALL
SoftClient4ES	1	8,192 MB	1 / 1	37.9 s
SoftClient4ES	2	4,096 MB	2 / 2	37.8 s
SoftClient4ES	3	2,730 MB	3 / 3	39.9 s
SoftClient4ES	4	2,048 MB	4 / 4	45.1 s
SoftClient4ES	5	1,638 MB	5 / 5	47.6 s
Trino (stock)	1	8,192 MB	0 / 1	killed

Outcome. In an 8 GB budget, SoftClient4ES completes five concurrent extractions — 50 million rows for 5× the work at +26% wall time — while a single Trino stock client cannot complete one. This measures *client-side* capacity; it does not measure server-side concurrency, which is a separate question this benchmark does not address.

6 Cross-index JOIN

A join between a 1M-row index (`bench_1m`) and the 10M-row index, landed as a pandas DataFrame on both sides. Both engines execute the join server-side (SoftClient4ES in an embedded DuckDB, Trino in its own engine).

```
-- J0 plain join (1,000,000 rows out)
SELECT a.id, b.amount FROM bench_1m a JOIN bench_events_10m b ON a.id = b.id
-- J1 + predicate on the large leg (125,361 rows out)
SELECT a.id, b.amount FROM bench_1m a JOIN bench_events_10m b ON a.id = b.id WHERE b.status = 'paid'
-- J2 + GROUP BY (100 rows out)
SELECT b.category, COUNT(*), AVG(b.amount) FROM bench_1m a JOIN bench_events_10m b ON a.id = b.id GROUP BY b.category
```

SCENARIO	SOFTCLIENT4ES WALL	TRINO WALL	CLIENT CPU (SC4ES / TRINO)
J0 — plain join	27.6 s	26.3 s (1.05× faster)	0.06 s / 1.17 s
J1 — + WHERE	11.9 s	11.1 s (1.07× faster)	0.03 s / 0.21 s
J2 — + GROUP BY	29.7 s	27.3 s (1.09× faster)	0.02 s / 0.09 s

J0 and J2 are medians of 12 runs; J1 of 5.

Outcome. Trino is marginally faster on wall clock across all three joins (5–9%), reached on a single-shard index that does not exercise its split parallelism. SoftClient4ES does 5–20× less client-side work in every case. The marginal cost of adding a `GROUP BY` to the join is higher for SoftClient4ES (+2.1 s vs +0.9 s), so on join-side aggregation Trino is the more efficient engine.

7 Index topology sensitivity

Everything above was measured on a single-shard index. Trino's Elasticsearch connector creates one split per shard, so a single-shard index gives it a single reader — which invites the fair question of whether these results survive an index that lets Trino parallelise, read by a Trino cluster rather than one node.

They do, and the extraction gap widens.

Setup. Same host, same corpus regenerated from the same seed into a **5-shard** index, one segment per shard. Trino runs as a real cluster — a dedicated coordinator plus two workers, with `node-scheduler.include-coordinator=false` so the coordinator plans and serves results while the workers scan. Trino is deliberately given **6 CPU / 8 GB against SoftClient4ES's 4 CPU / 4 GB**: 1.5× the CPU and 2× the memory. Medians of 5 runs after 2 warm-ups. Trino's plan used **5 scan splits across its two workers**, read back from `system.runtime.tasks`, so the parallelism under test was genuinely exercised.

METRIC	SOFTCLIENT4ES		TRINO	
	1 SHARD	5 SHARDS	1 SHARD	5 SHARDS
S1 wall	37.1 s	27.8 s	53.1 s	44.7 s
S1 client CPU	4.6 s	3.8 s	25.1 s	24.1 s
S1 peak client memory	900 MB	919 MB	4,472 MB	4,466 MB
S1 ES wire	2,494 MB	2,553 MB	2,926 MB	2,949 MB
S3 wall (<code>GROUP BY</code> , 100 rows)	0.04 s	0.04 s	27.0 s	5.8 s
S3 ES wire	0 MB	0 MB	1,394 MB	1,419 MB
S5 — 10M rows in a 2 GB container	completes	completes	killed	killed

Extraction. Both engines benefit from the extra shards, and SoftClient4ES benefits more: 25% faster against Trino's 16%. The S1 ratio **widens from 1.43× to 1.60×**, on a topology chosen to favour Trino and with Trino holding half again our CPU.

Client cost is a property of the protocol, not the topology. Peak client memory moves by 2% on our side and 0.1% on Trino's; client CPU is flat to slightly lower for both. Sharding the index and adding Trino nodes does not change what the client pays, because the client is one process consuming one wire format however large the cluster is. The constrained-container result is unchanged with it: 10M rows still land as a DataFrame in 2 GB on one side and are still OOM-killed on the other.

Pushdown is architectural. The `GROUP BY` still moves **0 bytes off the cluster against 1,419 MB**. Five shards and two extra Trino nodes do not change it, because the connector pushes no aggregation down: it reads all 10 million rows whatever the topology, and merely reads them faster.

Where Trino gains most

Its aggregation wall-clock improves **4.7×** (27.0 s → 5.8 s), by far the largest single improvement measured in this benchmark for either engine. Scan parallelism is exactly what a multi-shard index gives Trino, and on aggregate-heavy work it converts directly into wall-clock.

8 Where Trino is stronger

A fair benchmark names the other system's strengths.

- **Aggregation over a sharded index** — given shards to parallelise over, Trino's `GROUP BY` wall-clock improves **4.7×** (27.0 s → 5.8 s on a 5-shard index), the largest single gain either engine showed in this benchmark. It still moves 1.4 GB off the cluster to get there.
- **Cross-index joins** — Trino is marginally faster on wall clock (J0–J2), and more efficient at aggregating over a join.
- **A lower-memory Arrow client exists** — via `connectorx`, Trino can land an Arrow *table* in less client memory than SoftClient4ES (609 MB vs 900 MB on S1), and with less CPU on the polars destination. (This does not extend to the full DataFrame, where SoftClient4ES uses less — S5.)
- **Distributed execution, spill-to-disk and fault tolerance** — Trino scales a single query across a cluster and spills to disk; this single-node benchmark does not exercise any of it.
- **Connector breadth** — Trino federates 40+ data sources; SoftClient4ES is Elasticsearch-focused.
- **Licensing** — Trino is Apache-2.0 throughout; SoftClient4ES gates result-set size by licence tier (see Reproduction).

9 Reproduction

Prerequisites: Docker (VM ≥ 12 GB RAM / ≥ 12 CPUs), a native CPython 3.12, and ≥ 32 GB host RAM — the client runs on the host, and Trino's stock client materialises 10M rows into Python objects.

```
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt

# Bring up Elasticsearch and load the data (deterministic; ~3 minutes)
docker compose up -d elasticsearch
.venv/bin/python generator/generate_and_load.py

# Bring up both engines
docker compose up -d flight-sql trino

# Extraction matrix S0-S4 (medians of 5 runs after 2 warm-ups)
.venv/bin/python runners/orchestrate.py --stop-idle-engine

# Constrained-memory (S5) and concurrency (S6)
.venv/bin/python runners/orchestrate_capped.py
.venv/bin/python runners/orchestrate_concurrent.py --budget 8

# Cross-index JOIN (J0-J2)
.venv/bin/python runners/orchestrate_join.py
```

Results are written under `results/<session>/`, one JSON per run, with the environment and the sidecar image digest captured automatically.

Licence. SoftClient4ES enforces a result-set quota by licence tier (Community 10,000 / Pro 1,000,000 / Enterprise unlimited), so the full 10M rows (S1/S2) run under an Enterprise-tier licence. **The licence lifts a row quota only — it does not change the batch size, the serialization, or the data path.** The runners assert exactly 10,000,000 rows, so a quota-capped run aborts rather than being reported — a truncated result can never be published by accident. With no licence the harness runs on the Community tier, which reproduces every scenario's shape at reduced (`--limit`) scale.

A Appendix — Methodology

What this benchmark measures — and what it does not

The data path under test:

```
Elasticsearch → row-serialized documents out of the cluster
  → SoftClient4ES: converted to Arrow ONCE at the sidecar, streamed as
    Arrow batches, consumed by the client with no further deserialization
  → Trino: parsed into Trino's columnar pages, re-serialized to Trino's
    row-oriented client protocol, re-parsed value-by-value in the client
```

Every client pays the Elasticsearch serialization leg. Elasticsearch hands external clients row-serialized documents (JSON by default; the SQL endpoint can also emit CSV/TSV/CBOR/Smile). None is columnar, and this benchmark never claims to avoid that step. What it measures is what happens *after* it: SoftClient4ES serializes to Arrow once and the client consumes that columnar buffer directly, while Trino re-serializes to a row protocol the client must parse again.

So this benchmark measures **large result-set extraction and client-side consumption** — wall-clock, client CPU, peak client memory — which is where the client representation dominates. It does **not** measure Elasticsearch-side query execution (I/O, scoring), where the wire format is irrelevant.

Fairness rules

- **Same host, same session, same index.** Both systems read `bench_events_10m` back to back.
- **Identical container limits:** Elasticsearch, the sidecar and Trino each get 4 CPU / 4 GB. Elasticsearch heap is pinned at 2 GB; Trino's official image auto-sizes its heap from container memory.
- **Page-size symmetry:** the sidecar's `ARROW_BATCH_SIZE=1000` equals Trino's `elasticsearch.scroll-size=1000`. Both are product defaults, pinned explicitly so the setting can be checked.
- **Authentication disabled on both paths,** so authentication cost is zero and identical on both sides.
- **Both clients are used the way their projects document them.** SoftClient4ES is driven with the standard `adbc_driver_flightsql` Arrow Flight SQL driver; Trino with its own `trino` Python package, and — for the fairness comparison in S1/S1r — with its fastest available clients (connectorx and the ADBC Trino driver). No SoftClient4ES-specific client code is used.
- **Trino uses its default type mapping.** `legacy_primitive_types` would return unparsed strings, deferring the parse rather than avoiding it, and is not used.
- **Warm cache, fresh process.** Each scenario's warm-ups run immediately before its measured runs, so both systems measure against a warm Elasticsearch page cache. Every measured run is a fresh OS process, so peak memory cannot leak between runs. Because SoftClient4ES runs first in each scenario, Trino meets a slightly warmer cache — an effect in Trino's favour.
- **Correctness before timing.** Each run asserts the exact expected row count; a run that returns the wrong count is discarded, never reported. Python is never run with `-O`, so the asserts always execute.
- The exact configuration lives alongside the harness, and the data generator is deterministic (fixed seed).

Metrics

- **Wall-clock:** `time.perf_counter()` around connect → materialized result. Connection setup is included because it is time the user waits. (The Flight runner dials the sidecar by IP literal to avoid a Go-driver DNS

lookup that would otherwise add a fixed per-connection latency; real deployments should reuse connections.)

- **Client CPU:** `time.process_time()` for the client process — the CPU spent turning the wire format into usable values.
- **Peak client memory:** the process's peak physical footprint. On macOS this is the kernel's `ri_lifetime_max_phys_footprint`, which includes compressed pages and is therefore immune to the macOS memory compressor (plain RSS under-reports under memory pressure). On Linux the harness falls back to `ru_maxrss`.
- **ES wire bytes:** read from the container network counters, so a wire-volume difference is measured, not asserted.

Interpretation guardrails

- **S1 / S1r / S2** differences are attributable to the wire format and client-side representation.
- **S3** differences are attributable to **aggregation pushdown**, not the wire format: SoftClient4ES compiles the `GROUP BY` into an Elasticsearch aggregation, while Trino's connector performs predicate pushdown only and scans all rows. S3 is never a wire-format result.
- **S4** is the control: with a small result the wire format stops mattering, and near-parity there is a sign the benchmark is honest, not a weakness.
- **Where Trino is stronger** is published alongside the results (section 8).

Known limitations

- **Single node throughout.** This does not exercise Trino's distributed execution, spill-to-disk or fault tolerance — real strengths of Trino that a single-node extraction benchmark cannot show.
- **One flat mapping, no nested fields or arrays.** This avoids Trino's documented gaps in those types, so the connector is not handicapped; it also means the benchmark says nothing about SQL coverage, only about extraction efficiency.
- **Single primary shard in the main results — and that setup favours SoftClient4ES on wall-clock.** Trino's connector creates one split per shard, so a single-shard index gives it one reader and its scan parallelism is not exercised. Note what bounds this: the *shard* count, not the node count — additional Trino workers cannot split a single reader.

This is no longer left as a caveat: **section 7 publishes the measured sensitivity run** — the same corpus regenerated from the same seed into a 5-shard index, read by a real 3-node Trino cluster given 6 CPU / 8 GB against our 4 CPU / 4 GB. Both engines get faster; the S1 gap widens from 1.43× to 1.60×; client CPU, peak client memory and the 2 GB container threshold do not move; the `GROUP BY` still moves 0 bytes against 1.4 GB. Trino's own largest gain in the whole benchmark appears there too — its aggregation wall-clock improves 4.7× — and is published as such.

The distinction the run establishes: **wall-clock is topology-sensitive; client cost and pushdown are not.** The client is one process consuming one wire format however large the cluster, and the connector pushes no aggregation down at any shard count. That is now measured rather than argued.

- **Trino's spooling protocol was not benchmarked separately.** That variant is superseded by the S1/S1r fairness comparison against connectorx and the ADBC Trino driver, which measure Trino's fastest *columnar* clients directly — a stronger fairness position than the spooling protocol alone.

B Appendix — Issues found and fixed while benchmarking

Building this benchmark surfaced several issues in SoftClient4ES. All were fixed, and the results in this report were measured on the released [0.2.5](#) build that contains the fixes.

Extraction performance

- **Deep-paging sort regression (Elasticsearch 8+).** For a query with no `ORDER BY`, the streaming pager injected `_shard_doc` as the primary sort. From Elasticsearch 8 / Lucene 9 onward this defeats the document-id skip optimisation, so each page re-scanned the whole index instead of seeking to the next position — about **60 ms/page instead of 8 ms/page** on the 10M-row index, an effect that grows with index size. The fix uses `_doc` as the sort under a point-in-time context (which Elasticsearch turns into a total order via an automatic tiebreaker), restoring efficient paging while keeping results complete. A multi-shard completeness test guards against regression.
- **Per-row conversion cost.** The row-to-Arrow path did redundant per-row work (repeated map normalization and re-parsing of each response page). This was reduced to a single parse per page and a single-pass row conversion, which is the main reason the client-CPU figures in S1/S2 are as low as they are.

Result completeness

A family of cases silently returned incomplete results and were corrected:

- `GROUP BY` without `LIMIT` returned only the first ~10 groups (the default aggregation size).
- Window `PARTITION BY` truncated to the first ~10 partitions.
- `SELECT` without `LIMIT` on the non-scroll path returned only the first ~10 rows.
- An explicit `LIMIT` above `index.max_result_window` failed while the same query with no `LIMIT` succeeded.

Each now returns the complete result set, and each is covered by a multi-shard completeness test. These matter for a benchmark because a silently truncated result would otherwise look like a fast, correct run.

Result shape

Extraction rows previously carried Elasticsearch hit-metadata columns (`_id`, `_index`, `_score`) in addition to the selected columns. Rows now contain exactly the columns the SQL projects, so the client receives — and pays for — only the requested data (8 columns for the S1 query).

Environment note

The published sidecar image originally shipped without a logging configuration, which caused it to log at DEBUG and write every Elasticsearch response byte to the container log — a large performance and data-exposure problem. The image now ships a logging configuration that keeps wire-level logging off by default; the benchmark runs the sidecar as shipped.